

Laboratorium nr 2 – Administracja BD

Monitorowanie liczby rekordów w bazach danych

Paweł Myszka
Nr albumu: 331720

27 października 2025

1 Cel laboratorium

Celem laboratorium było stworzenie systemu monitorowania liczby rekordów w tabelach użytkownika. System składa się z:

- Bazy administracyjnej przechowującej historie pomiarów
- Procedur automatyzujących proces zbierania danych
- Procedur analitycznych do wyświetlania historii i statystyk przyrostów

2 Zadanie 1 – Tworzenie baz danych

```
1 DECLARE @i INT = 1;
2 WHILE @i <= 5
3 BEGIN
4 DECLARE @dbname SYSNAME = N'PM_DB' + CAST(@i AS NVARCHAR(4));
5 IF DB_ID(@dbname) IS NULL
6 BEGIN
7 DECLARE @sql NVARCHAR(MAX) =
8 N'CREATE DATABASE ' + QUOTENAME(@dbname) +
9 N'_ON (NAME=N'' + @dbname + N'_Data'',
10 FILENAME=N''D:\stud\sem5\AdminBazDa\2\'
11 + @dbname + N'_Data.mdf'',
12 SIZE=20MB, MAXSIZE=200MB, FILEGROWTH=10MB)';
13 EXEC(@sql);
14 END
15 SET @i = @i + 1;
16 END
```

Utworzone bazy: PM_DB1, PM_DB2, PM_DB3, PM_DB4, PM_DB5.

3 Zadanie 2 – Utworzenie tabel w bazach

W bazach PM_DB1, PM_DB2, PM_DB3 utworzono struktury tabel:

- WOJ – województwa (3 rekordy)
- MIASTA – miasta (4 rekordy)
- OSOBY – osoby (5 rekordów bazowych)
- ETATY – etaty (4 rekordy bazowe)

Wykorzystano procedurę `pr_seed_pm331720`, która dynamicznie tworzy tabele i wypełnia je danymi.

4 Zadanie 3 – Dodanie rekordów w PM_DB3

W bazie PM_DB3 dodano dodatkowe rekordy:

```
1  USE PM_DB3;
2  INSERT INTO dbo.OSOBY(id_miasta, imie, nazwisko)
3  VALUES
4  ((SELECT TOP 1 id_miasta FROM dbo.MIASTA
5   WHERE nazwa='WARSZAWA'),'Piotr','G rski'),
6  ((SELECT TOP 1 id_miasta FROM dbo.MIASTA
7   WHERE nazwa='WARSZAWA'),'Agnieszka','Le na');
8
9  -- Dodanie 3 dodatkowych etat w
10 DECLARE @idp INT = (SELECT TOP 1 id_osoby FROM dbo.OSOBY
11 WHERE imie='Piotr' AND nazwisko='G rski');
12 DECLARE @ida INT = (SELECT TOP 1 id_osoby FROM dbo.OSOBY
13 WHERE imie='Agnieszka' AND nazwisko='Le na');
14
15 INSERT INTO dbo.ETATY(id_osoby, stanowisko, pensja, [od])
16 VALUES
17 (@idp, 'Kierownik', 11000, '20240101'),
18 (@idp, 'Koordynator', 8000, '20230101'),
19 (@ida, 'M odszy Specjalista', 4200, '20230215');
```

Po dodaniu rekordów:

- OSOBY: $5 + 2 = 7$ rekordów
- ETATY: $4 + 3 = 7$ rekordów

5 Zadanie 4 – Śledzenie liczby rekordów

5.1 4.1 – Utworzenie tabel administracyjnych

Utworzono bazę B25_ADM_PM331720 z dwiema tabelami:

```
1  USE B25_ADM_PM331720;
2  GO
3
4  CREATE TABLE dbo.DB_CHECK (
5   check_id INT IDENTITY(1,1) PRIMARY KEY,
6   db_nam NVARCHAR(100) NOT NULL,
7   d_stamp DATETIME NOT NULL DEFAULT GETDATE(),
8   opis NVARCHAR(200) NOT NULL,
9   [usr] NVARCHAR(128) NOT NULL DEFAULT USER_NAME(),
10  [s_usr] NVARCHAR(128) NOT NULL DEFAULT SUSER_NAME()
11 );
12
13 CREATE TABLE dbo.DB_CHECK_ITEMS (
14  check_id INT NOT NULL,
15  tb_nam NVARCHAR(256) NOT NULL,
16  liczba_reordow INT NOT NULL,
17  tb_check_d_stamp DATETIME NOT NULL DEFAULT GETDATE(),
18  CONSTRAINT FK_DB_CHECK_ITEMS__DB_CHECK
19  FOREIGN KEY (check_id) REFERENCES dbo.DB_CHECK(check_id)
20 );
```

Struktura danych:

- DB_CHECK – główna tabela z informacją o wykonaniu snapshotu (kto, kiedy, która baza)
- DB_CHECK_ITEMS – szczegóły: liczba rekordów w każdej tabeli dla danego snapshotu

5.2 4.2 – Procedura monitorująca pojedynczą bazę

Procedura `pr_pm_check_one` dla podanej bazy:

1. Wstawia rekord do `DB_CHECK` i pobiera `check_id`
2. Iteruje po wszystkich tabelach użytkownika
3. Zlicza rekordy używając `sys.partitions`
4. Zapisuje wyniki w `DB_CHECK_ITEMS`

```
1 CREATE PROCEDURE dbo.pr_pm_check_one
2 @db sysname,
3 @opis NVARCHAR(200) = N'PM331720 - snapshot pojedynczej bazy'
4 AS
5 BEGIN
6 SET NOCOUNT ON;
7
8 -- Walidacja istnienia bazy
9 IF DB_ID(@db) IS NULL
10 BEGIN
11 RAISERROR(N'Baza %s nie istnieje.',16,1,@db);
12 RETURN;
13 END
14
15 -- Wstawienie rekordu z informacją o snapshocie
16 INSERT INTO dbo.DB_CHECK(db_nam, opis) VALUES (@db, @opis);
17 DECLARE @check_id INT = SCOPE_IDENTITY();
18
19 -- Dynamiczne SQL zliczające rekordy we wszystkich tabelach
20 DECLARE @sql NVARCHAR(MAX) = N'
21 INSERT INTO dbo.DB_CHECK_ITEMS(check_id, tb_nam, liczba_rekordow)
22 SELECT ' + CAST(@check_id AS NVARCHAR(12)) + N',
23 QUOTENAME(s.name) + ''.'' + QUOTENAME(t.name),
24 CAST(SUM(CASE WHEN p.index_id IN (0,1)
25 THEN p.[rows] ELSE 0 END) AS INT)
26 FROM ' + QUOTENAME(@db) + N'.sys.tables t
27 JOIN ' + QUOTENAME(@db) + N'.sys.schemas s
28 ON s.schema_id = t.schema_id
29 JOIN ' + QUOTENAME(@db) + N'.sys.partitions p
30 ON p.[object_id] = t.[object_id]
31 WHERE t.is_ms_shipped = 0
32 GROUP BY s.name, t.name
33 ORDER BY s.name, t.name;';
34
35 EXEC sys.sp_executesql @sql;
36 END
```

Uwaga: Użyto `sys.partitions` zamiast `COUNT(*)` dla wydajności – nie wymaga skanowania całej tabeli.

5.2.1 Przykład użycia

```
1 EXEC dbo.pr_pm_check_one @db = N'PM_DB1';
```

5.2.2 Wyniki działania

Tabela `DB_CHECK`:

1	check_id	db_nam	d_stamp	opis
2	-----	-----	-----	-----
3	1	PM_DB1	2025-10-27 21:03:50.553	PM331720 - snapshot ...

Tabela DB.CHECK_ITEMS:

1	check_id	tb_nam	liczba_reordow
2	-----	-----	-----
3	1	[dbo].[WOJ]	3
4	1	[dbo].[MIASTA]	4
5	1	[dbo].[OSOBY]	5
6	1	[dbo].[ETATY]	4

5.3 4.3 – Procedura monitorująca wszystkie bazy

Procedura `pr_pm_check_all` używa kursora do iteracji po wszystkich bazach użytkownika (z wyłączeniem systemowych i administracyjnej):

```

1 CREATE PROCEDURE dbo.pr_pm_check_all
2 @opis NVARCHAR(200) = N'PM331720-snapshotwszystkichuserDB'
3 AS
4 BEGIN
5 SET NOCOUNT ON;
6 DECLARE @db sysname;
7
8 -- Kursor po bazach u ytkownika
9 DECLARE cur CURSOR LOCAL FAST_FORWARD FOR
10 SELECT name FROM sys.databases
11 WHERE name NOT IN (N'master', N'model', N'msdb',
12 N'tempdb', N'B25_ADM_PM331720')
13 AND state = 0 AND is_read_only = 0
14 AND name LIKE N'PM-DB%';
15
16 OPEN cur;
17 FETCH NEXT FROM cur INTO @db;
18
19 WHILE @@FETCH_STATUS = 0
20 BEGIN
21 BEGIN TRY
22 -- Wywo anie procedury dla pojedynczej bazy
23 EXEC dbo.pr_pm_check_one @db=@db, @opis=@opis;
24 PRINT N'Zliczono tabelewbazie: ' + @db;
25 END TRY
26 BEGIN CATCH
27 PRINT N'Pomini to baz  ' + @db + N': ' + ERROR_MESSAGE();
28 END CATCH;
29
30 FETCH NEXT FROM cur INTO @db;
31 END
32
33 CLOSE cur;
34 DEALLOCATE cur;
35 END

```

Logika działania:

1. Kursor iteruje po wszystkich bazach z prefiksem PM_DB
2. Dla każdej bazy wywołuje `pr_pm_check_one`
3. Obsługa błędów (TRY-CATCH) – jeśli baza jest niedostępna, procedura kontynuuje prace
4. Informuje o sukcesie/porażce dla każdej bazy

5.3.1 Przykład użycia

```
1 EXEC dbo.pr_pm_check_all;
```

5.3.2 Wyniki

Komunikaty w konsoli:

```
1 Zliczono tabele w bazie: PM_DB1
2 Zliczono tabele w bazie: PM_DB2
3 Zliczono tabele w bazie: PM_DB3
```

Dane w DB_CHECK_ITEMS (fragment):

check_id	tb_nam	liczba_reordow
2	[dbo].[WOJ]	3
2	[dbo].[MIASTA]	4
2	[dbo].[OSOBY]	5
2	[dbo].[ETATY]	4
3	[dbo].[WOJ]	3
3	[dbo].[MIASTA]	4
3	[dbo].[OSOBY]	7
3	[dbo].[ETATY]	7

5.4 4.4 – Procedura wyświetlająca historie tabeli

Procedura pr_pm_history wyświetla wszystkie pomiary dla podanej tabeli w podanej bazie:

```
1 CREATE PROCEDURE dbo.pr_pm_history
2   @db sysname,
3   @table NVARCHAR(256)
4 AS
5 BEGIN
6   SET NOCOUNT ON;
7
8   -- Normalizacja nazwy tabeli (usunięcie nawiasów w kwadratowych)
9   DECLARE @t_norm NVARCHAR(256) =
10     REPLACE(REPLACE(@table, N'[', N''), N']', N'');
11
12   -- Parsowanie schematu i nazwy tabeli
13   DECLARE @schema SYSNAME, @tab SYSNAME;
14   IF CHARINDEX('.', @t_norm) > 0
15     BEGIN
16       SET @schema = PARSENAME(@t_norm, 2);
17       SET @tab = PARSENAME(@t_norm, 1);
18     END
19   ELSE
20     BEGIN
21       SET @schema = N'dbo';
22       SET @tab = @t_norm;
23     END
24
25   -- Wyszukanie wszystkich pomiarów z różnymi wariantami nazw
26   DECLARE @tb_quoted NVARCHAR(300) =
27     QUOTENAME(@schema) + N'.' + QUOTENAME(@tab);
28   DECLARE @tab_only_q NVARCHAR(300) = QUOTENAME(@tab);
29   DECLARE @tb_plain NVARCHAR(300) = @schema + N'.' + @tab;
30
31   SELECT c.db_nam, i.tb_nam, c.check_id,
32     c.d_stamp, i.liczba_reordow
33   FROM dbo.DB_CHECK c
```

```

34 JOIN dbo.DB_CHECK_ITEMS i ON i.check_id = c.check_id
35 WHERE c.db_nam = @db
36 AND (i.tb_nam = @tb_quoted
37 OR i.tb_nam = @tab_only_q
38 OR REPLACE(REPLACE(i.tb_nam, N'[', N'''), N']', N''))
39 IN (@tb_plain, @tab))
40 ORDER BY c.d_stamp, c.check_id;
41
42 IF @@ROWCOUNT = 0
43 RAISERROR(N'Brak historii dla bazy %s i tabeli %s',
44 16, 1, @db, @table);
45 END

```

5.4.1 Przykład użycia

```

1 EXEC dbo.pr_pm_history @db='PM_DB3', @table='OSOBY';

```

5.4.2 Przykładowe wyniki

db_nam	tb_nam	check_id	d_stamp	liczba_reordow
PM_DB3	[dbo].[OSOBY]	5	2025-10-27 20:15:30.120	5
PM_DB3	[dbo].[OSOBY]	8	2025-10-27 21:10:45.333	7
PM_DB3	[dbo].[OSOBY]	12	2025-10-27 22:05:12.887	7

Historia pokazuje, że w bazie PM_DB3 tabela OSOBY początkowo miała 5 rekordów, a po dodaniu dwóch osób (Piotr Górski, Agnieszka Leśna) ma 7 rekordów.

5.5 4.5 – Procedura porównująca dwa snapshoty

Procedura pr_pm_stats porównuje dwa pomiary i oblicza przyrosty:

```

1 CREATE PROCEDURE dbo.pr_pm_stats
2 @check_id_1 INT,
3 @check_id_2 INT
4 AS
5 BEGIN
6 SET NOCOUNT ON;
7
8 -- a) Walidacja check_id
9 IF NOT EXISTS(SELECT 1 FROM dbo.DB_CHECK
10 WHERE check_id = @check_id_1)
11 OR NOT EXISTS(SELECT 1 FROM dbo.DB_CHECK
12 WHERE check_id = @check_id_2)
13 BEGIN
14 RAISERROR(N'Check_id nie istnieje w B25-ADM-PM331720.dbo.DB_CHECK',
15 16, 3);
16 RETURN;
17 END
18
19 -- b) Po czenie danych z dw ch pomiar w i obliczenie przyrost w
20 ;WITH c1 AS (
21 SELECT c.db_nam, i.tb_nam, i.liczba_reordow
22 FROM dbo.DB_CHECK c
23 JOIN dbo.DB_CHECK_ITEMS i ON i.check_id = c.check_id
24 WHERE c.check_id = @check_id_1
25 ),
26 c2 AS (
27 SELECT c.db_nam, i.tb_nam, i.liczba_reordow
28 FROM dbo.DB_CHECK c

```

```

29 JOIN dbo.DB_CHECK_ITEMS i ON i.check_id = c.check_id
30 WHERE c.check_id = @check_id_2
31 ),
32 j AS (
33 SELECT COALESCE(c2.db_nam, c1.db_nam) AS baza,
34 COALESCE(c2.tb_nam, c1.tb_nam) AS tabela,
35 ISNULL(c1.liczba_reordow, 0) AS liczba_reordow_dla_check_1,
36 ISNULL(c2.liczba_reordow, 0) AS liczba_reordow_dla_check_2,
37 ISNULL(c2.liczba_reordow, 0) - ISNULL(c1.liczba_reordow, 0)
38 AS przyrost
39 FROM c1 FULL OUTER JOIN c2
40 ON c1.db_nam = c2.db_nam AND c1.tb_nam = c2.tb_nam
41 )
42 -- Zwracanie szczeg. owej tabeli z przyrostami
43 SELECT baza, tabela, liczba_reordow_dla_check_1,
44 liczba_reordow_dla_check_2, przyrost
45 FROM j
46 ORDER BY baza, tabela;
47
48 ;WITH mx AS (
49 SELECT baza, tabela, przyrost,
50 ROW_NUMBER() OVER (PARTITION BY baza
51 ORDER BY przyrost DESC, tabela) AS rn
52 FROM j
53 )
54 SELECT baza, tabela AS najwiekszy_przyrost_w_bazie_pomiedzy_check1_i_check2
55 FROM mx
56 WHERE rn = 1
57 ORDER BY baza;
58 END

```

Kluczowe elementy:

- **FULL OUTER JOIN** – obsługuje przypadki gdy tabela istniała tylko w jednym pomiarze
- **ISNULL** – zabezpieczenie przed NULL przy obliczaniu przyrostu
- **ROW_NUMBER()** – wybór tabeli z największym przyrostem w każdej bazie

5.5.1 Przykład użycia

```

1 -- Porównanie pomiarów przed i po dodaniu rekordów
2 EXEC dbo.pr_pm_stats @check_id_1=5, @check_id_2=8;

```

5.5.2 Przykładowe wyniki

Wynik 1 – Szczegółowe przyrosty dla wszystkich tabel:

	baza	tabela	check_1	check_2	przyrost
3	PM_DB3	[dbo].[ETATY]	4	7	3
4	PM_DB3	[dbo].[MIASTA]	4	4	0
5	PM_DB3	[dbo].[OSOBY]	5	7	2
6	PM_DB3	[dbo].[WOJ]	3	3	0

Wynik 2 – Największy przyrost w każdej bazie:

	baza	najwiekszy_przyrost_w_bazie_pomiedzy_check1_i_check2
3	PM_DB3	[dbo].[ETATY]

Wyniki pokazują, że w bazie PM_DB3 największy przyrost (+3) wystąpił w tabeli ETATY.

5.6 4.5.1 – Statystyki miesięczne

Procedura ST_M_PM331720 wykorzystuje procedure pr_pm_stats do analizy przyrostów w danym miesiącu:

```
1 CREATE PROCEDURE dbo.ST_M_PM331720
2 @parametr_ym NCHAR(6), -- format: YYYYMM np. '202510'
3 @baza NVARCHAR(100)
4 AS
5 BEGIN
6 SET NOCOUNT ON;
7
8 -- Wyszukanie najwcześniejszego pomiaru w miesi cu
9 DECLARE @c1 INT = (
10 SELECT MIN(check_id) FROM dbo.DB_CHECK
11 WHERE CONVERT(NCHAR(6), d_stamp, 112) = @parametr_ym
12 AND db_nam = @baza
13 );
14
15 -- Wyszukanie najpóźniejszego pomiaru w miesi cu
16 DECLARE @c2 INT = (
17 SELECT MAX(check_id) FROM dbo.DB_CHECK
18 WHERE CONVERT(NCHAR(6), d_stamp, 112) = @parametr_ym
19 AND db_nam = @baza
20 );
21
22 -- Walidacja
23 IF @c1 IS NULL OR @c2 IS NULL
24 BEGIN
25 RAISERROR(N'Brak pomiaru w dla wskazanego miesi ca i bazy.', 16, 6);
26 RETURN;
27 END
28
29 -- Wywołanie procedury porównawczej
30 EXEC dbo.pr_pm_stats @check_id_1 = @c1, @check_id_2 = @c2;
31 END
```

Format daty: CONVERT(NCHAR(6), d_stamp, 112) zwraca YYYYMM (np. 202510 dla października 2025)

5.6.1 Przykład użycia

```
1 -- Przyrosty w październiku 2025 dla PM_DB3
2 EXEC dbo.ST_M_PM331720 @parametr_ym=N'202510', @baza=N'PM_DB3';
```

5.7 4.5.2 – Statystyki dzienne

Analogiczna implementacja dla statystyk dziennych:

```
1 CREATE PROCEDURE dbo.ST_D_PM331720
2 @parametr_dzien NCHAR(8), -- format: YYYYMMDD np. '20251027'
3 @baza NVARCHAR(100)
4 AS
5 BEGIN
6 SET NOCOUNT ON;
7
8 -- Najwcześniejszy pomiar w dniu
9 DECLARE @c1 INT = (
10 SELECT MIN(check_id) FROM dbo.DB_CHECK
11 WHERE CONVERT(NCHAR(8), d_stamp, 112) = @parametr_dzien
12 AND db_nam = @baza
13 );
```



```
14
15 -- Najp mniejszy pomiar w dniu
16 DECLARE @c2 INT = (
17 SELECT MAX(check_id) FROM dbo.DB_CHECK
18 WHERE CONVERT(NCHAR(8), d_stamp, 112) = @parametr_dzien
19 AND db_nam = @baza
20 );
21
22 -- Walidacja
23 IF @c1 IS NULL OR @c2 IS NULL
24 BEGIN
25 RAISERROR(N'Brak pomiar w dla wskazanego dnia i bazy.', 16, 7);
26 RETURN;
27 END
28
29 -- Wywołanie procedury porównawczej
30 EXEC dbo.pr_pm_stats @check_id_1 = @c1, @check_id_2 = @c2;
31 END
```

Format daty: CONVERT(NCHAR(8), d_stamp, 112) zwraca YYYYMMDD (np. 20251027)

5.7.1 Przykład użycia

```
1 -- Przyrosty w dniu 27 października 2025 dla PM_DB3
2 EXEC dbo.ST_D_PM331720 @parametr_dzien=N'20251027', @baza=N'PM_DB3';
```

6 Dodatkowe elementy implementacji

6.1 Backup bazy administracyjnej

Automatyczny backup z timestampem w nazwie pliku:

```
1 DECLARE @bakfile NVARCHAR(260) =
2 N'D:\stud\sem_5\AdminBazDa\2\B25_ADM_PM331720_backup_' +
3 REPLACE(CONVERT(NVARCHAR(20), GETDATE(), 120), ':', '-') + N'.bak';
4
5 BACKUP DATABASE [B25_ADM_PM331720]
6 TO DISK = @bakfile
7 WITH INIT, FORMAT,
8 NAME = N'Full_backup_B25_ADM_PM331720-PM331720';
```

Nazwa pliku np: B25_ADM_PM331720_backup_2025-10-27 22-30-15.bak

6.2 Utworzenie dedykowanego użytkownika

```
1 -- Login na poziomie serwera
2 CREATE LOGIN pm_331720_login WITH PASSWORD = N'Pm331720@2025';
3
4 -- Użytkownik w bazie administracyjnej
5 USE B25_ADM_PM331720;
6 CREATE USER pm_331720_user FOR LOGIN pm_331720_login;
7
8 -- Uprawnienia: odczyt i zapis
9 ALTER ROLE db_datareader ADD MEMBER pm_331720_user;
10 ALTER ROLE db_datawriter ADD MEMBER pm_331720_user;
```

7 Scenariusz testowy

7.1 Krok 1: Pierwszy snapshot

```
1 EXEC dbo.pr_pm_check_all @opis=N'Snapshot_pocz tkowy';
```

Stan przed dodaniem rekordów:

- PM_DB1: OSOBY=5, ETATY=4
- PM_DB2: OSOBY=5, ETATY=4
- PM_DB3: OSOBY=5, ETATY=4

7.2 Krok 2: Dodanie rekordów w PM_DB3

```
1 USE PM_DB3;  
2 INSERT INTO dbo.OSOBY VALUES (...); -- +2 osoby  
3 INSERT INTO dbo.ETATY VALUES (...); -- +3 etaty
```

7.3 Krok 3: Drugi snapshot

```
1 EXEC dbo.pr_pm_check_all @opis=N'Po_dodaniu_rekord w';
```

Stan po dodaniu:

- PM_DB1: OSOBY=5, ETATY=4 (bez zmian)
- PM_DB2: OSOBY=5, ETATY=4 (bez zmian)
- PM_DB3: OSOBY=7, ETATY=7 (przyrost!)

7.4 Krok 4: Analiza przyrostów

```
1 -- Por wnanie check_id=5 i check_id=8  
2 EXEC dbo.pr_pm_stats @check_id_1=5, @check_id_2=8;  
3 -- Historia tabeli OSOBY w PM_DB3  
4 EXEC dbo.pr_pm_history @db='PM_DB3', @table='OSOBY';  
5 -- Statystyki miesi czne  
6 EXEC dbo.ST_M_PM331720 @parametr_ym=N'202510', @baza=N'PM_DB3';
```

8 Podsumowanie

W ramach laboratorium zrealizowano kompleksowy system monitorowania liczby rekordów:

*Paweł Myszka, nr albumu 331720
27 października 2025*